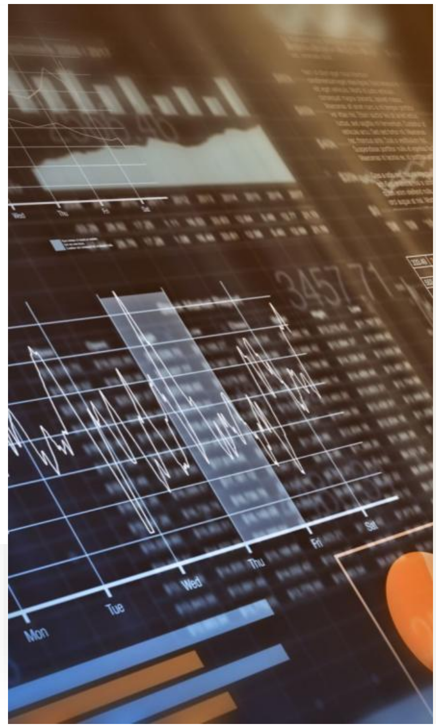
Business Data Analytics – Lecture 3: Data Quality & Feature Engineering

Assoc. Prof. Nguyen Binh Minh, Ph.D.



1

Learning Objectives (CLO)

- Understand data quality dimensions and measurement.
- Diagnose and handle missingness.
- Detect and avoid data leakage.
- Encode categorical variables and scale numeric features.
- Engineer features and perform feature selection.
- Build reproducible, leakage-safe pipelines.

2

Agenda & Timing

1) Data quality (40')
2) Missing data (35')
3) Data leakage (25')
4) Categorical encoding (35')
5) Scaling & outliers (25')
6) Feature engineering & selection (40')
7) Pipeline & mini-case (30')
8) Short quiz & wrap-up (10')

3

Running Case: Telco Churn Prediction

Dataset: 10k customers, 35 variables (demographics, plan, usage, label).

Goal: predict churn in the next 30 days.

Challenges: missing income/tenure, leakage (post-event features),

High cardinality plan_id, mixed scales.

4

What is Data Quality?

- Accuracy, Completeness, Consistency.
- Validity, Timeliness, Provenance.
- Direct impact on model performance and ops cost.

5

Data Quality Scorecard

Example KPIs: % missing < 2%/feature; duplicate rate < 0.5%; out-of-range < 0.1%.

Data SLA: daily H+1 before 08:00; stable schema outside maintenance windows.

6

Data Quality Lifecycle

Ingest → Profile → Clean → Validate → Monitor → Feedback.

Combine rule-based checks with descriptive statistics.

7

Quick Data Profiling

Distributions, min/max, outliers, cardinality, missing map.

Visuals: histogram, boxplot, correlation heatmap, category bars.

8

Duplicates & Orphan Records

- Dedup: exact keys vs. fuzzy (edit distance).
- Orphans: broken foreign keys → integrity checks.

9

Units & Value Domains

- Normalize currencies, percentages, and units (km ↔ m).
- Validity rules (e.g., age ∈ [0, 120]).

10

Missingness Mechanisms

- MCAR: completely at random.
- MAR: depends on observed variables.
- MNAR: depends on the missing variable itself.
- Impact: biased estimates, lower power, model bias.

11

Diagnosing Missingness

- Column/row missing maps; relation between missingness and label.
- Little’s MCAR idea; compare distributions in missing vs. non-missing groups.

12

Example: Missing by Segment

Income missing more in Prepaid; Tenure missing in new customers.

Likely MAR by plan_type; possible MNAR for income.

13

Missingness Strategies — Overview

- Row/column drop if excessive missingness or low-value feature.
- Imputation: mean/median/mode, group-wise, kNN, MICE, model-based, business rules.
- Add an is_missing indicator.

14

Simple & Group-wise Imputation

Median/Mode robust to outliers.

Group-wise medians by plan_type.

15

kNN Imputation

- Find k nearest neighbors → average/majority.
- Pros: preserves multivariate structure; Cons: compute-heavy, scale-sensitive.
- Apply scaling inside a pipeline to avoid leakage.

16

MICE (Multiple Imputation by Chained Equations)

- Model each incomplete feature iteratively until convergence.
- Produce multiple datasets and pool estimates.
- Best for MAR; preserves correlation structure.

17

Business- and Time-aware Imputation

- Time series: forward/backward fill, interpolation, seasonal Kalman.
- Domain rules: income = base salary × factor; apply caps.

18

Assessing Imputation Impact

- Compare model metrics (AUC/RMSE) across strategies.
- Check distribution drift after imputation; keep predictive signal.

19

Common Pitfalls in Missingness

- Global-mean imputation using full data → leakage.
- Forgetting the is_missing flag.
- Imputing before train/test split → leakage!

20

Data Leakage — Definition & Impact

- Information from future/test creeps into training.
- Over-optimistic training scores, sharp drop in production.

21

Leakage Types

- Target leakage: features contain the label or its consequences.
- Train–test contamination: preprocessing fitted on full data.
- Temporal leakage: using post-hoc features for forecasting.

22

Example of Target Leakage (Churn)

- Feature 'refund_in_30d' appears in training but occurs after churn.
- Mitigation: drop or time-shift to before the prediction point.

23

Avoiding Temporal Leakage

- Time-based splits (TimeSeriesSplit/rolling origin).
- Freeze feature windows to pre-T0 data only.

24

Anti-Leakage Pipeline Principles

- Fit preprocessing only on training folds.
- Use Pipeline/ColumnTransformer.
- Perform cross-validation on the full pipeline.

25

Data Splitting Strategies

- Holdout, KFold, StratifiedKFold.
- TimeSeriesSplit for temporal data.
- Group-aware split when multiple rows per customer.

26

Categorical Encoding — Overview

- One-Hot, Ordinal, Target/Mean, WoE, Hashing, Frequency/Count.
- Issues: high cardinality, rare categories, target-encoding leakage.

27

One-Hot Encoding (OHE)

- Pros: simple, no order assumption.
- Cons: dimensionality explosion with high cardinality.
- Tips: group rare levels to 'Other'; use hashing if needed.

28

Ordinal Encoding

Map categories to integers with true ordering (S < M < L < XL).

Avoid for unordered categories.

29

Target/Mean Encoding (with CV)

- Replace category with target mean (with smoothing).
- Compute inside CV loops to prevent leakage.
- Add noise/smoothing to reduce overfit.

30

Weight of Evidence (WoE)

- WoE = ln((Good_i/Bad_i)/(Good/Bad)) — widely used in credit risk.
- Pros: monotonicity, interpretability; Cons: requires binning and enough samples.

31

Hashing Trick

- Map categories to a fixed space via hashing.
- Pros: handles high cardinality and streaming; Cons: collisions, less interpretable.

32

High-Cardinality Tactics

Collapse rare levels; frequency/count encoding.

CV-safe target encoding; or hashing + embeddings.

33

Text Features (quick)

Bag-of-Words/TF-IDF; n-grams; remove stopwords.

Pair with linear/logistic models or tree-based learners.

34

Entity Embeddings (idea)

- Learn dense representations of categories with neural nets.
- Great for high-cardinality interactions.

35

Why & When to Scale

- Scale-sensitive algorithms: kNN, k-means, SVM, PCA, L1/L2 regression.
- Tree-based models less sensitive but scaling helps when mixing models.

36

Scaling Techniques

- StandardScaler: $(x-\mu)/\sigma$.
- MinMaxScaler: [0,1].
- RobustScaler: median/IQR for outlier robustness.
- Always fit on train, transform train/test.

37

Power / Variance-Stabilizing Transforms

- Log, Box-Cox ($x>0$), Yeo-Johnson ($x\in\mathbb{R}$).
- Make distributions closer to normal; reduce skew.

38

Quantile / Rank-Gauss Transform

- Map to uniform/normal distributions.
- Useful for linear models and normality assumptions.

39

Outliers — Detection

- Z-scores, IQR (Q1−1.5*IQR, Q3+1.5*IQR).
- Model-based: Isolation Forest, LOF.
- Note: outliers may carry business signal.

40

Outliers — Handling

- Capping/Winsorization; log transforms; robust loss.
- Conditional removal with domain investigation.

41

Feature Transformations Toolkit

- Binning (equal-width/frequency), interactions, polynomials.
- Ratios/Rates (e.g., ARPU = revenue/usage).
- Time-normalized rates.

42

Date–Time Features

- Calendar: day/week/month/quarter/year; holiday flags.
- Seasonality with sine–cosine for cycles (hour/day).
- Recency/freshness: days since last activity.

43

Geo Features (if coordinates)

- Haversine distances, regional clustering, point density.
- Grid encoding (geohash) for regional aggregation.

44

Aggregation / Group-by Features

- Counts/frequencies per user/plan.
- Group means/medians/std; shares/ratios.
- Leave-one-out strategies to reduce leakage.

45

Time-Series Features

- Lags (t−1, t−7, t−30), rolling mean/std/max/min.
- Exponential moving features; differencing.

46

Text/NLP Features (extended)

- Sentiment scores, keyword flags (complaints).
- TF-IDF + SVD (LSA) for dimension reduction.

47

Feature Selection — Overview

- Filter: Chi-square, ANOVA, Mutual Information.
- Wrapper: RFE, SFS/SBS.
- Embedded: L1/L2, tree-based importance, SHAP.

48

Filter Methods

- Chi-square for categorical vs. target.
- ANOVA F-test for numeric vs. class.
- Mutual Information for non-linear dependencies.

49

Wrapper Methods

- RFE with estimator; choose k via CV.
- Trade-off: accuracy vs. compute time.

50

Embedded & Regularization

- Lasso (L1) for sparsity; Ridge (L2) for variance reduction.
- Elastic Net balances L1/L2.
- Tree-based importance: Gini/Permutation.

51

Dimensionality Reduction

- PCA preserves variance in orthogonal components.
- t-SNE/UMAP for visualization; avoid direct forecasting use.

52

Golden Rules for Pipelines

Fit	Never fit preprocessing on the full dataset.
Use	Use ColumnTransformer for per-type pipelines.
Wrap	Wrap hyperparameter search around the pipeline.

53

Pipeline Diagram (concept)

- Raw → Split → (Impute → Encode → Scale → FeatureSelect) → Model → Eval.
- For temporal data: enforce pre-T0 feature windows.

54

Sklearn Pipeline (leakage-safe) — Pseudocode

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, RobustScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from category_encoders.target_encoder import TargetEncoder

num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", RobustScaler())
])

cat_high_pipe = Pipeline([
    ("target_enc", TargetEncoder())
])

cat_low_pipe = Pipeline([
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

pre = ColumnTransformer([
    ("num", num_pipe, num_cols),
    ("cat_high", cat_high_pipe, high_card_cols),
    ("cat_low", cat_low_pipe, low_card_cols)
])

clf = Pipeline([
    ("preprocess", pre),
    ("model", LogisticRegression(max_iter=200))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
score = cross_val_score(clf, X, y, cv=cv, scoring="roc_auc")
```

55

Model Evaluation after Preprocessing

- Metrics: ROC-AUC/PR-AUC (class imbalance), Brier score, KS.
- Compare impute/encode/scale strategies via consistent CV.

56

Data Validation & Monitoring

- Schema contracts (types, ranges, constraints).
- Pre-deploy checks: null %, ranges, cardinality, drift.
- Production monitoring with alerts and thresholds.

57

Data Contracts & Schema Evolution

- Agreements between data producers and consumers: fields, definitions, SLAs.
- Schema versioning and backfill strategies.

58

Reproducibility & Experiment Tracking

- Random seeds, environment files.
- Track params, metrics, artifacts, and data snapshots.

59

Common Anti-patterns

- Future-derived features; preprocessing on full data.
- Unbounded one-hot; ignoring rare categories.
- No is_missing flags; ignoring group-aware splits.

60

Mini-case: Churn — Part 1 (Profiling)

- Task: profile and define a data quality checklist.
- Deliverable: DQ metrics table for top 10 columns.

61

Mini-case: Churn — Part 2 (Missing & Encoding)

- Choose strategies for income and tenure imputation.
- Encode plan_id (high cardinality) with CV-safe target encoding.

62

Mini-case: Churn — Part 3 (Scaling & Model)

- Compare Logistic (scaled) vs. XGBoost (no scaling needed).
- Report ROC-AUC/PR-AUC and calibration plot.

63

In-class Hands-on

- 1) Build the leakage-safe pipeline as shown.
- 2) Run 5-fold CV and log results.
- 3) Write five lines summarizing trade-offs.

64

Summary — Key Takeaways

- Data quality sets the ceiling for model performance.
- Leakage-safe pipelines are mandatory.
- Encoding/Scaling must match the model context.
- Feature engineering blends domain knowledge and controlled experiments.

65

Appendix — Formulas & Notes

- Standardize: $z = (x - \mu) / \sigma$.
- $IQR = Q3 - Q1$; capping at $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$.
- WoE/IV basics; time cyclic encoding: $\sin(2\pi t/T)$, $\cos(2\pi t/T)$.

66

Appendix — Group-wise Imputation (pandas)

- X['income'] = X.groupby('plan_type')['income'].transform(
-     lambda s: s.fillna(s.median())
-)
- X['income_is_missing'] = X['income'].isna().astype(int)

67

Appendix — CV-safe Target Encoding

```
from category_encoders.target_encoder import TargetEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression

te = TargetEncoder(cols=['plan_id'], smoothing=0.2)
model = Pipeline([
   ('pre', ColumnTransformer([('plan', te, ['plan_id'])], remainder='drop')),
   ('clf', LogisticRegression(max_iter=200))
])
# Use within cross_val_score/cross_val_predict to avoid leakage
```

68

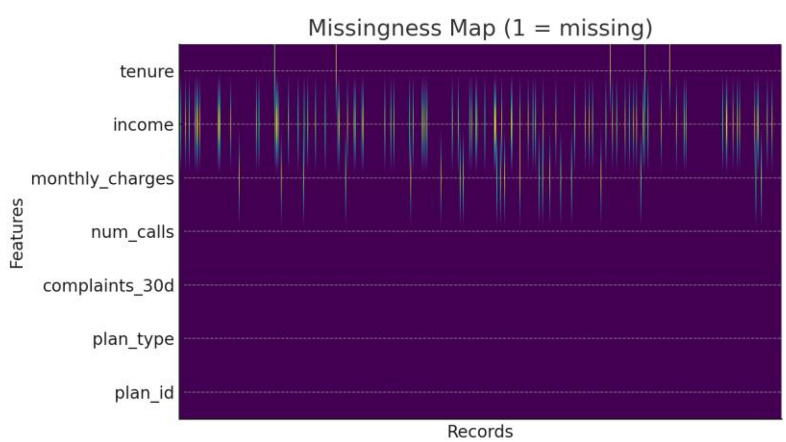
Recommended References

- Kuhn & Johnson (2019) — Feature Engineering and Selection.
- Pang-Ning Tan et al. — Introduction to Data Mining.
- scikit-learn docs: Pipeline, ColumnTransformer, Imputation, Encoding, Scaling.
- Credit scoring resources: monotone binning, WoE/IV.

69

Visual Example — Missingness Map

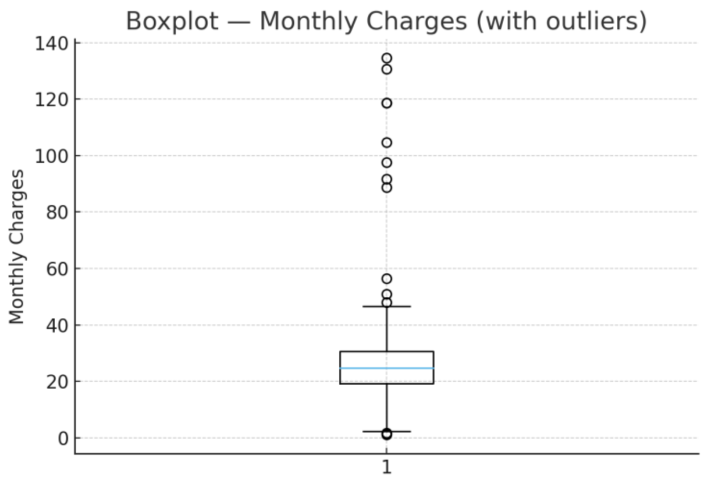
Synthetic telco-like dataset (1 = missing)



70

Visual Example — Outliers

Boxplot of Monthly Charges



71